

Coding Principles - Expense Splitter

1. SOLID Principles

- S - Single Responsibility Principle: fiecare clasă din backend are o responsabilitate clară (ex: UserController gestionează doar cererile legate de utilizatori).
- O - Open/Closed Principle: clasele pot fi extinse fără a fi modificate (ex: metodele din serviciile backend pot fi suprascrise sau extinse dacă este necesar).
- L - Liskov Substitution Principle: nu folosim moștenire excesivă, dar păstrăm coerența între tipuri și instanțe.
- I - Interface Segregation Principle: folosim interfețe separate în repository-uri pentru diferite tipuri de date.
- D - Dependency Inversion Principle: logica este separată de persistarea datelor folosind Spring Data JPA.

2. Best Practices

- Denumiri semnificative pentru clase, variabile și metode.
- Separarea clară a responsabilităților: controllers, services, repositories.
- Folosirea adnotărilor de validare în backend (ex: @NotBlank, @Email, @Size).
- Folosirea tipurilor în TypeScript pentru frontend.
- Folosirea unui fișier de tipuri centralizat (`types.ts`).
- Organizarea codului pe module în React Native: screens, services, components.

3. Securitate și validare

- Autentificare cu JWT.
- Validări pe ambele părți (frontend și backend).
- Tratarea cazurilor de eroare cu mesaje clare și coduri HTTP adecvate.